

Rhizome: Publishing Repository Knowledge as Open Knowledge Format Bundles for Coding Agents

Parser-built dependency graphs, Leiden subsystems, and reviewable context that co-evolves with the code

Venkat Malviya

Independent researcher, Bengaluru, India · September 2026

ABSTRACT. Coding agents depend on context about the repository they change, and most systems supply it by chunking source files into a vector index and retrieving the chunks most similar to the task. This discards the structure that matters for software change (who depends on whom, which files form a subsystem, which callers are exposed to a vulnerable function) and stores the result in a private, opaque index that drifts from the code. We present *Rhizome*, a repository scanner that publishes code knowledge as an *Open Knowledge Format* (OKF) bundle: a directory of markdown concepts with YAML frontmatter, committed next to the code. Rhizome (i) builds an exact, weighted import graph with a deterministic parser rather than LLM extraction; (ii) discovers subsystems with the Leiden algorithm, whose communities are guaranteed to be connected, and materialises them as the bundle's navigation hierarchy; (iii) defines an OKF *code profile* that types markdown links by section so that dependency semantics survive in a vendor-neutral format; (iv) makes knowledge content-addressed and incrementally updated, so that a pull request carries its knowledge diff and CI can reject stale knowledge; and (v) retrieves context with category-aware graph traversals for feature, bug, security and refactor tasks. We evaluate a proof-of-concept on six open-source Python repositories of 37 to 2,931 files. Scans are deterministic, take 0.3–14 s, and a one-file change rewrites two documents. Leiden produced no disconnected communities in 60 runs while Louvain produced one, and Leiden matched or exceeded Louvain's modularity on every repository. In a co-change benchmark built from 927 real future commits, the import graph is a far better relatedness signal than directory layout (mean recall@5 0.460 vs 0.293). Lexical similarity is the strongest single signal (0.513), and the graph is complementary to it: rank fusion reaches 0.520, and the union of both top-10 lists recovers 2–10 points of recall that lexical retrieval alone misses. We argue that the main value of the graph is structural and explanatory — typed, reviewable reasons for why a file is in context — rather than raw recall, and we release the design as a profile that any producer or consumer can implement.

Keywords: coding agents, retrieval-augmented generation, knowledge graphs, community detection, Leiden algorithm, Open Knowledge Format, software architecture recovery

1 Introduction

Large language models can write and modify code, but they know nothing about a particular private repository: what a module is for, which files a change will affect, where authentication is enforced, or which requirement a function implements. That knowledge has to be supplied as context at the moment the agent acts. The dominant way of doing so is retrieval-augmented generation (RAG) [5]: split files into chunks, embed them, store them in a vector index, and return the top-*k* chunks most similar to the task description.

Similarity search works when the relevant code *looks like* the request. Software change often does not. A refactor must reach every transitive caller of the function being changed; a security fix must consider every path by which untrusted input reaches a vulnerable call; a new feature must follow the conventions of the subsystem it joins. These are structural questions about relationships, and

GraphRAG [4] has shown for prose corpora that making relationships explicit, and summarising groups of related entities, helps answer questions that top- k similarity cannot.

Three further problems are specific to code-context systems as they are commonly built:

1. **Opacity.** The knowledge lives in a private index (vectors, a database, a vendor's graph schema). Humans cannot read or review it, and other tools cannot consume it.
2. **Drift.** Code changes on every commit. Re-indexing is typically periodic and all-or-nothing, and nothing ties a code change to the corresponding knowledge change.
3. **Misplaced LLM effort.** For prose, an LLM must read text to find entities and relations. For code, the relations (imports, calls, definitions) can be extracted exactly and cheaply by a parser. Spending LLM tokens to approximate them adds cost and error.

The Open Knowledge Format (OKF), published by Google Cloud in June 2026 [6], formalises the "LLM wiki" pattern [7] as a vendor-neutral format: a bundle is a directory of markdown files with YAML frontmatter, each file a concept, linked by ordinary markdown links. OKF deliberately requires only a type field and leaves the content model to producers. It has been demonstrated on data catalogues (tables, datasets, metrics). As far as we are aware, it has not been applied to source code.

This paper applies it to source code. Our contributions are:

1. A **design for publishing repository knowledge as an OKF bundle**, in which structure is extracted deterministically and LLMs are used, optionally, only for natural-language descriptions (Section 4).
2. The **OKF code profile**: concept types, required fields, a path-based identity rule, and *section-typed links* that preserve dependency semantics in plain markdown (Section 4.5).
3. **Leiden communities materialised as navigation**: a two-level, deterministic, identity-stable community hierarchy over the import graph that becomes the bundle's progressive-disclosure structure (Section 4.4).
4. A **co-evolution protocol**: content-addressed concepts, content-change timestamps, an append-only change log, and a CI check that fails a pull request whose code and knowledge disagree (Section 4.6).
5. **Category-aware retrieval** over the published bundle, with a different graph walk for feature, bug, security and refactor tasks, and requirement documents linked into the same graph (Sections 4.7–4.8).
6. A **proof-of-concept implementation** and an **evaluation** on six repositories, including a co-change benchmark with no train/test leakage (Sections 5–6).

Individually, several ingredients exist: parser-built code graphs exposed to agents [8, 9], community detection on code graphs [9, 8], and software clustering for architecture recovery [15, 16]. The contribution here is the combination, and in particular publishing the result in an open, reviewable format that co-evolves with the code through ordinary code review.

2 Background

2.1 Retrieval-augmented generation and its limits

Baseline RAG retrieves by semantic or lexical similarity between a query and chunks of text [5]. It struggles with *multi-hop* questions, whose answer requires following a chain of relationships across

passages that do not individually resemble the query, and with *global* questions about a whole corpus, whose answer is not contained in any single chunk [4]. Both kinds are common in software change. "What breaks if I change this signature?" is multi-hop. "How is authentication structured?" is global.

2.2 GraphRAG

GraphRAG [4, 12] indexes a text corpus in four stages. It segments the corpus into text units, uses an LLM to extract entities, relationships and claims, clusters the resulting graph hierarchically with the Leiden algorithm, and generates bottom-up summaries for each community. At query time, *global search* map-reduces over community summaries, and *local search* uses vector similarity to find entry entities and then expands along the graph. The index is stored as tables (entities, relationships, communities, reports, text units), by default as parquet files, with embeddings in a separate vector store. GraphRAG targets unstructured prose. Its extraction is probabilistic and costly, and its outputs are machine-oriented.

2.3 Community detection: Louvain and Leiden

A community is a set of nodes more densely connected to each other than to the rest of the graph. Modularity [3] scores a partition by comparing each community's internal edge weight with the weight expected under a degree-preserving random graph, with a resolution parameter γ controlling granularity. Louvain [2] optimises modularity by repeatedly moving single nodes to the neighbouring community that most improves the score, and then aggregating communities into super-nodes. Traag et al. [1] showed that Louvain can yield arbitrarily badly connected, even internally disconnected, communities. A node that bridges two parts of its community can be moved out, leaving the rest locally optimal but disconnected, and aggregation then freezes the defect. On real networks they found up to 25% of Louvain communities badly connected and up to 16% disconnected. The Leiden algorithm adds a refinement phase between local moving and aggregation, which splits each community into well-connected sub-communities before aggregating, and uses a fast local-move queue. It guarantees that communities are connected after every iteration and, when iterated, converges to partitions in which all subsets of communities are locally optimally assigned. It is also faster than Louvain on large graphs.

For a code-knowledge system this guarantee is not cosmetic. A community is summarised and presented as a subsystem. A disconnected "subsystem" would be summarised as one unit while actually being two unrelated groups of files, and the summary would mislead both humans and agents.

2.4 Open Knowledge Format

An OKF v0.1 bundle [6] is a directory of markdown documents with YAML frontmatter. Each document is a concept whose file path is its identity. The only required field is `type`. The recommended fields are `title`, `description`, `resource` (pointer to the real artefact), `tags` and `timestamp`. Concepts link to each other with markdown links, which turns the directory into a graph. Optional `index.md` files support progressive disclosure, and `log.md` files record history. OKF's stated principles are minimal opinionation, producer/consumer independence, and being a format rather than a platform.

3 Related Work

Code graphs for agents. Codebase-Memory [8] builds a tree-sitter knowledge graph over 66 languages, stores it in SQLite, partitions call graphs into communities with Louvain, and exposes 14

query tools over the Model Context Protocol. In its evaluation, agents using the graph answered with somewhat lower quality than agents reading files directly (83% vs 92%) but used roughly ten times fewer tokens. Repowise [9] builds a tree-sitter dependency graph, applies Leiden and PageRank to find subsystems and hotspots, regenerates documentation on every commit, and serves graph queries to agents over MCP. Rhizome shares the parser-first stance of both. It differs in publishing an open, human-reviewable format rather than a tool-specific store, in using community structure as the navigation hierarchy of that format, in tying knowledge updates to code review, in linking requirements into the same graph, and in category-specific retrieval.

GraphRAG and LLM wikis. GraphRAG [4] contributes the community-summary idea and the local-search pattern (vector entry, then graph expansion), both of which Rhizome adopts. Rhizome replaces LLM extraction with parsing, because code relations are explicit. LLM wikis and convention files such as `AGENTS.md` give agents curated markdown to read before working [7]. OKF standardises that pattern [6], and Rhizome generates such a wiki for code automatically.

Software clustering and architecture recovery. Clustering module dependency graphs to recover subsystems has a long history, e.g. Bunch [15], which searches for modularisation-quality-maximising partitions, and ACDC [16], which uses comprehension-driven patterns. Community detection has also been applied to decompose monoliths into microservices. Rhizome uses clustering for a different end: as the organising structure of machine- and human-readable context, with connectivity guarantees and stable identities across versions.

Co-change. Mining version histories for files that change together [11] is an established signal for guiding changes. We use it as *ground truth* for evaluating context retrieval, not as a feature.

4 Design

4.1 Principles

1. **Deterministic structure, LLM for prose.** Everything a parser can know exactly (files, symbols, imports, reference counts, security-relevant call patterns) is extracted without an LLM. LLMs, if used, write only descriptions, and results are cached by content hash.
2. **Format, not platform.** The published artefact is an OKF bundle. Indexes (vector, lexical, graph databases) are derived, disposable views.
3. **Identity follows the code, not the analysis.** A file's concept path derives from its repository path, never from its community, because communities change as code evolves.
4. **Freshness is content-addressed.** A concept is stale if and only if its content hash differs from the file. Git metadata, which changes when the bundle itself is committed, is not used as a freshness signal.
5. **Knowledge changes are reviewed like code.** The bundle lives in the repository, and a change and its knowledge update travel in the same pull request.

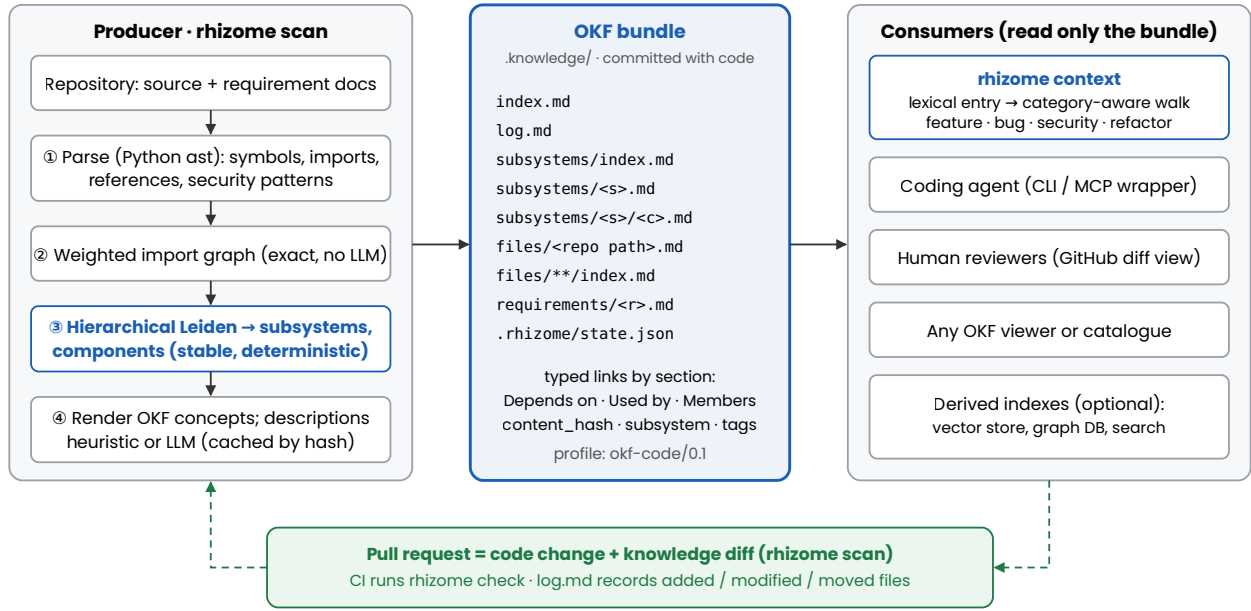


Figure 1. Rhizome architecture. The producer extracts structure deterministically, detects subsystems with Leiden and publishes an OKF bundle into the repository. Consumers read only the bundle. Code changes and knowledge changes travel together in a pull request, and CI rejects stale knowledge.

4.2 Parsing

Each source file is parsed with the language's own parser (the proof of concept uses Python's `ast`). The parser records the module's canonical dotted name and aliases (e.g. with and without a `src/` prefix), top-level classes and functions with docstrings and line numbers, all imports with relative imports resolved against the package, a count of every referenced name, and security-relevant patterns: dynamic code execution, shell execution, unsafe deserialisation, YAML loading without a safe loader, weak hashes, disabled TLS verification, SQL built by string formatting, and string literals assigned to secret-like names. Files that fail to parse are kept as nodes with the error recorded.

4.3 Weighted import graph

Imports are resolved to repository files by trying submodule names first (`from pkg import mod`) and then the longest module prefix. Unresolved imports are recorded as external dependencies. The graph G has an edge $a \rightarrow b$ when file a imports file b , labelled with the imported names N_{ab} and weighted by use:

$$w(a, b) = 1 + \min(50, \sum_{n \in N_{ab}} \text{refs}_a(n))$$

where $\text{refs}_a(n)$ counts name and attribute references to n in a . The cap stops one heavily used utility from dominating. Community detection uses the undirected graph U with $w(a,b)+w(b,a)$ on each edge.

4.4 Hierarchical Leiden with stable identities

Subsystems are found by running Leiden [1] with the modularity quality function (`RBConfiguration`, $\gamma = 1$) on the subgraph of files that have at least one internal dependency edge, iterated until a stable iteration so that its connectivity guarantees hold. Files with no internal edges (scripts, examples, empty package initialisers) would otherwise each form a singleton community. They are instead grouped by top-level directory into explicitly labelled *unlinked* groups, recorded with grouping:

directory so that no consumer mistakes them for detected structure. Any subsystem larger than a size threshold (default 15 files) is partitioned again with Leiden on its induced subgraph to form *components*, and singleton components are merged into the component they are most strongly connected to. Each community records its hub (highest weighted PageRank [14]), its members ranked by centrality, its cohesion (internal weight divided by internal plus boundary weight), and whether it is connected.

Two engineering details turned out to be essential. **Stable identity.** Community slugs are derived from the common directory and hub file, and on re-scan each new community is matched to a previous one when their member sets have Jaccard similarity ≥ 0.5 , reusing its slug. A small code change therefore does not rename subsystems or rewrite their concepts. **Determinism.** Community detection is sensitive to node order, and graph libraries commonly iterate node sets whose order depends on a per-process hash seed. Without canonical ordering, our first implementation produced different partitions in different processes of the same repository, which would make any CI freshness check fail spuriously. Rhizome canonicalises node and edge order before detection and breaks every centrality tie by path. We verified byte-identical bundles (excluding timestamps) across hash seeds on Celery.

4.5 The OKF code profile

OKF deliberately does not define a content model. The profile `okf-code/0.1` defines one for code (Table 1), and a bundle declares it in its root `index.md`.

type	one per	profile fields beyond OKF's recommended set
Source File	source file	language, content_hash, subsystem; optional component, loc, fan_in, fan_out, c
Subsystem	level-1 community	level, size, hub, grouping, cohesion, connected
Component	level-2 community	as Subsystem, plus parent
Requirement	requirement document	—
Index / Log	directory / bundle	—

Table 1. Concept types of the OKF code profile. Tags include the language, `subsystem:<slug>`, and optionally `test`, `area:<name>` and one `security:<rule>` per non-informational finding.

Section-typed links. Markdown links carry no relation type, but dependency semantics are the point of a code graph. The profile therefore types a link by the heading of the section it appears in. In a Source File, links under *# Depends on* mean "imports", links under *# Used by* mean "is imported by", and each line lists the imported names. In a Subsystem, *# Members* lists files by centrality, and *# Depends on subsystems* aggregates cross-community edges. A consumer can reconstruct the typed graph from any conformant bundle without the producer's internal state. The result is still ordinary markdown that renders on any code host. An abbreviated concept:

```

---
type: Source File
title: shop/auth.py
description: Password hashing and session tokens.
resource: https://github.com/acme/shop/blob/HEAD/shop/auth.py
tags: [python, 'subsystem:api', 'area:auth', 'security:hardcoded-secret', 'security:weak-hash']
timestamp: '2026-09-25T20:26:57+00:00'
language: python
content_hash: sha256:fcd9a5eb8f1562e4
subsystem: api
fan_in: 1
fan_out: 1
---
# Subsystem
Part of [shop/api.py cluster](/subsystems/api.md).
# Depends on
- [shop/db.py](/files/shop/db.py.md): uses `find_user`
# Used by
- [shop/api.py](/files/shop/api.py.md): uses `login`
# Security notes
- L5 · high · hardcoded-secret: string literal assigned to `SECRET_KEY`
- L8 · medium · weak-hash: `hashlib.md5()`

```

Progressive disclosure. The root `index.md` gives an overview, the subsystem list with one-line summaries, and security hotspots. `subsystems/index.md` and per-subsystem component indexes form the next levels, and `files/**/index.md` mirrors the directory tree. An agent can therefore descend from architecture to file in three or four reads without loading the whole bundle. Because the hierarchy comes from Leiden on the import graph rather than from folders, it reflects actual coupling.

4.6 Co-evolution protocol

Every Source File concept carries `content_hash`, a truncated SHA-256 of the file bytes. A scan re-parses the repository, recomputes the graph and communities (cheap and deterministic), renders every concept, and writes only those whose bytes changed. The timestamp of a concept is the time its *content* last changed. It is carried forward from producer state when the rendered content is unchanged, so an unrelated commit produces no diff in untouched concepts. The `resource` field uses a branch-stable URL rather than a commit SHA, for the same reason. Each scan that changes the bundle appends an entry to `log.md` listing added, modified and removed files, files that moved between subsystems, and new or dissolved communities. Expensive LLM descriptions are cached by content hash and member-hash keys and regenerated only for changed files and communities.

Protocol. (1) A change that edits source files updates the affected concepts in the same pull request by running `rhizome scan`. (2) CI runs `rhizome check`, a dry-run scan that exits non-zero if any concept would change. (3) Reviewers see the knowledge diff next to the code diff. Scan options are recorded in the bundle, so the check reproduces the producer exactly.

This turns knowledge maintenance, the step that causes human wikis to decay [7], into a mechanical, reviewable by-product of normal development.

4.7 Requirement linking

Requirement documents (markdown) become Requirement concepts. They are linked to code by exact mentions of repository paths or dotted module names, public symbol names of at least four characters, and distinctive file stems (generic stems such as `utils` or `core` are excluded). Links are bidirectional: `# Implemented by` in the requirement, and `# Requirements` in each file and aggregated

in its subsystem. The planner therefore sees *why* code exists in the same graph as *how* it is connected. The matching is deliberately conservative and explainable. Each link records what matched.

4.8 Category-aware retrieval

The consumer reads only the published bundle. It builds a BM25 [13] index over the title, description, tags and symbols of Source File and Requirement concepts, selects up to four *entry points* (a matched requirement contributes its implementing files; test files are excluded except for bug tasks), and then expands along section-typed links with a walk chosen by task category (Table 2). Every item in the resulting context pack carries a human-readable reason ("caller at distance 1 (exposure path)", "blast radius: depends on the change at distance 2"), and the pack is cut to a character budget in rank order.

Category	Graph walk after entry points
feature	dominant subsystem among entry points → its most central members; direct dependencies of entry points
bug	dependencies up to distance 2; tests that import the entry points
security	entry scores boosted by <code>security:*</code> and <code>area:auth</code> tags; callers up to distance 2 (exposure paths); flagged files in the same subsystem
refactor	transitive reverse dependencies up to distance 3 (blast radius)

Table 2. Retrieval strategies. Categories are supplied by the caller, typically from an agent's own task classification.

On Celery, for example, the security query "unsafe pickle deserialization of task messages" ranks `celery/backends/database/__init__.py` first. It is the one non-test module the scanner flagged for `pickle.loads`. The walk then adds its callers with exposure-path reasons. The refactor query "change the retry backoff in task retry" enters at `celery/app/autoretry.py` and returns its reverse dependencies with distances.

5 Implementation

The proof of concept is about 1,600 lines of Python. It depends on `networkx`, `python-igraph` with `leidenalg` (the reference Leiden implementation), and `PyYAML`, and it needs no network access or API key. It provides five commands: `scan` (produce or update a bundle; `--docs` links requirements; `--llm` uses a hosted model for descriptions, cached by hash), `check` (CI freshness gate), `validate` (OKF conformance: every document has a type and every link resolves), `context` (category-aware context pack as markdown or JSON), and `stats` (Leiden vs Louvain connectivity). A test suite covers import resolution, security rules, graph construction, community coverage and connectivity, conformance, incremental updates, timestamp stability, retrieval behaviour per category, deletions, and determinism across hash seeds. The parser is isolated behind a small interface so that further languages can be added, for example with `tree-sitter`.

6 Evaluation

We ask four questions. **RQ1:** Is scanning cheap enough to run on every change, and are bundles conformant? **RQ2:** Are updates incremental, idempotent and deterministic? **RQ3:** Does Leiden's connectivity guarantee matter on real import graphs? **RQ4:** How useful is graph structure, compared with lexical similarity and directory layout, for selecting the files a change actually touches?

Subjects. Six open-source Python projects at their current heads: Flask, Requests, HTTPX, Rich, Celery and Django. Django was used for scale only, as a shallow clone. All runs used a single CPU core of a cloud Linux container, with default options and no LLM.

6.1 RQ1–RQ2: cost, conformance, incrementality, determinism

Repository	files	import edges	subsystems	components	documents	bundle	full scan	security findings
Requests	37	106	8	0	55	110 KB	0.28 s	14
HTTPX	60	122	4	7	82	193 KB	0.38 s	10
Flask	83	179	11	3	129	242 KB	0.87 s	4
Rich	213	730	16	29	277	564 KB	1.07 s	7
Celery	432	1,289	22	61	596	1.3 MB	2.69 s	46
Django	2,931	9,089	32	176	3,840	8.3 MB	14.0 s	266

Table 3. Scan statistics at repository heads. Subsystem counts include unlinked directory groups (1–6 per repository). "Security findings" counts non-informational pattern matches, including in tests. Six Django test files did not parse under the Python 3.11 interpreter used (mostly newer syntax, one an intentional syntax-error fixture); they were kept as nodes with the error recorded.

Every bundle validated as conformant: all documents typed and all links resolving. A full scan of Django (2,931 files) took 14 s, of which 8 s was parsing. On every repository we then appended one function to a mid-ranked non-test file and re-scanned. **Exactly two documents** changed each time: that file's concept, and the root index, whose `source_digest` changed. No subsystem was renamed, no unrelated concept was rewritten, and a subsequent check reported the bundle fresh. Re-scan time, however, equalled full-scan time (0.26–14.6 s), because the proof of concept re-parses every file. Caching parse results by content hash is a straightforward optimisation that we have not yet implemented. Finally, scanning Celery in three processes with different hash seeds produced identical bundles apart from timestamps, and a check in a fourth process passed.

6.2 RQ3: Leiden vs Louvain on import graphs

Repository	nodes / edges	Leiden comm.	Louvain comm.	Leiden Q	Louvain Q	Leiden disc.	Louvain disc.
Requests	34 / 98	5.0	5.0	0.3160	0.3160	0	0
HTTPX	57 / 117	3.0	3.0	0.4805	0.4805	0	0
Flask	79 / 167	9.0	8.9	0.4771	0.4767	0	0
Rich	207 / 698	11.6	11.0	0.4331	0.4300	0	0
Celery	371 / 1,282	16.3	13.0	0.5870	0.5852	0	1
Django	2,191 / 9,015	29.8	23.2	0.5869	0.5815	0	0

Table 4. Ten seeds per algorithm on the linked part of each undirected import graph ($\gamma = 1$). Columns give mean number of communities, mean modularity Q, and total disconnected multi-node communities across the ten runs. Leiden via `leidenalg` (iterated to stability), Louvain via `networkx`.

Import graphs are sparse and fairly modular ($Q \approx 0.32$ – 0.59), so badly connected partitions are rarer here than in the dense social and citation networks studied in [1]. Louvain nonetheless produced a disconnected community on Celery in one of ten runs. Leiden produced none in sixty runs, as its guarantee requires. Leiden also matched or exceeded Louvain's modularity on every repository and found finer partitions on the larger ones (Celery +25%, Django +28% communities). The margins are small. The argument for Leiden in this setting is the guarantee: a subsystem concept is summarised and navigated as a unit, and the guarantee rules out a class of silently misleading summaries at no extra cost.

6.3 RQ4: co-change retrieval

Method. For each history-bearing repository, we walk back from the head through non-merge commits until we have collected 300 that touch between 2 and 10 Python files, and take the parent of the oldest as snapshot T. We build every representation at T only. For each later commit, restricted to files that existed at T (at least two), we treat each touched file as the *seed* of a task. The ground truth is the set of other files that the real commit touched. This mirrors the question an agent faces when starting from one relevant file ("what else will this change touch?"), and because the ground truth lies in the future relative to T, there is no leakage. Co-change is a well-established proxy for change coupling [11]. We score seven rankings of all other files at T, using $\text{recall}@k$ (fraction of ground truth in the top k) for $k \in \{5, 10\}$:

- **Random.**
- **Lexical:** cosine similarity of TF-IDF vectors over identifier-split tokens of whole files. This is our stand-in for similarity retrieval; we did not evaluate neural embeddings.
- **Same directory:** same-directory files first, each group ordered lexically. This is the folder-layout baseline.
- **Leiden community:** same component, then same subsystem, then the rest, each ordered lexically.
- **Graph:** undirected import distance up to 3, ties broken by edge weight and then lexically, followed by community members and the rest.
- **Interleaved:** alternating graph and lexical ranks.
- **Fusion:** reciprocal rank fusion of lexical and graph rankings [10].

Repository (snapshot)	files · commits · seeds	Random		Directory		Community		Graph		Lexical		Fusion	
		@5	@10	@5	@10	@5	@10	@5	@10	@5	@10	@5	@10
Flask (2018-01)	77 · 110 · 269	.065	.139	.326	.459	.341	.417	.423	.621	.573	.715	.527	.651
Requests (2013-10)	75 · 156 · 426	.064	.144	.584	.752	.389	.879	.526	.785	.609	.780	.602	.835
HTTPX (2020-01)	62 · 109 · 302	.090	.172	.283	.427	.365	.509	.364	.530	.372	.550	.381	.550
Rich (2021-11)	176 · 271 · 701	.024	.038	.153	.216	.434	.536	.513	.626	.462	.637	.546	.635
Celery (2024-01)	376 · 281 · 826	.014	.026	.120	.230	.473	.502	.476	.566	.551	.635	.546	.639
Mean	927 commits · 2,524 seeds	.051	.104	.293	.417	.400	.568	.460	.626	.513	.663	.520	.662

Table 5. Co-change recall@5 and @10. Bold marks the best of the columns shown. The interleaved variant (mean .494 / .662) is omitted for width but shown in Figure 2; it is slightly higher than the bold value for HTTPX @5 and Rich @10. Snapshots differ in age because repositories accumulate qualifying commits at different rates.

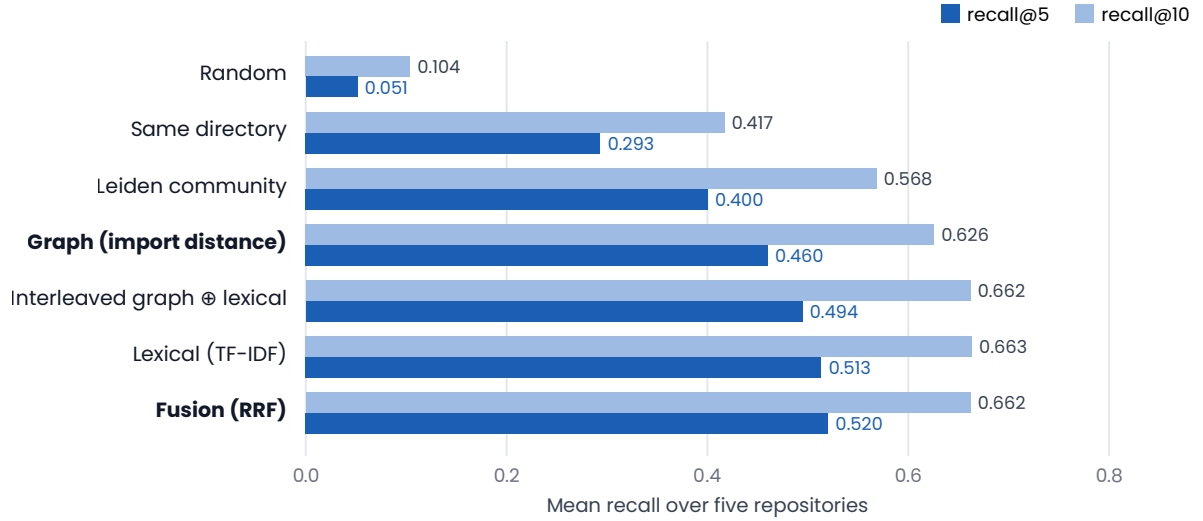


Figure 2. Mean co-change recall over five repositories. Graph structure clearly beats folder layout. Lexical similarity is the strongest single signal, and fusing it with the graph is on par with or slightly above lexical alone.

Repository	found only by graph@10	found only by lexical@10	lexical@10	either (union)	gain from graph
Flask	.022	.116	.715	.737	+2.2
Requests	.104	.099	.780	.884	+10.4
HTTPX	.047	.066	.550	.596	+4.6
Rich	.059	.070	.637	.696	+5.9
Celery	.040	.109	.635	.675	+4.0

Table 6. Complementarity: mean fraction of ground truth that only one method's top 10 contains, and recall of the union of both top-10 lists (up to 20 files) in points over lexical@10.

Findings.

1. **The import graph encodes coupling that folder layout does not.** Graph ranking beats same-directory ranking on all five repositories at recall@10 and on four of five at recall@5 (mean recall@5 .460 vs .293; on Celery .476 vs .120). The exception is Requests at @5, a small, flat package. This supports organising navigation by detected subsystems rather than by folders.
2. **Lexical similarity is the strongest single signal for co-change.** Compared with graph ranking, it wins at recall@5 on four of five repositories. Graph ranking wins on Rich (.513 vs .462), plausibly because its many small modules share less vocabulary with the files they change alongside.
3. **The two signals are complementary, but fusion gains are modest.** Each finds ground truth the other misses. The union of their top-10 lists recovers 2.2–10.4 points of recall beyond lexical@10. Rank fusion yields the best mean recall@5 (.520 vs .513) and ties at @10 (.662 vs .663), improving markedly on Requests (@10: .835 vs .780) and Rich (@5: .546 vs .462) and losing on Flask. We therefore do not claim that the graph improves raw recall on its own.
4. **Communities are a coarse signal.** Ranking community members first helps when communities are small (Requests @10: .879) but hurts at small k in large subsystems. This motivates using communities for orientation and summaries rather than as a ranking feature.

These results shaped the retrieval design in Section 4.8: lexical search selects entry points, and the graph supplies the *expansion and the reasons* (callers, blast radius, subsystem context), which similarity cannot provide and which a reviewer or agent can verify.

7 Discussion and Limitations

What the graph is for. Our strongest evidence for the graph is not recall but properties that similarity lacks: every context item has a typed, checkable reason; structure-dependent tasks (all callers of a function, all importers of a module) are answered exactly rather than approximately; and the discovered architecture gives agents and humans a map. The co-change benchmark measures one property, predicting what else will change, where content similarity is known to be strong, partly because tests and their subjects share vocabulary.

Threats to validity. (i) Co-change ground truth includes tangled commits and misses files that should have changed but did not. (ii) TF-IDF stands in for neural embeddings, which may perform differently. (iii) The subjects are five Python projects plus Django for scale. Results may differ for other languages, monorepos, or codebases built on dynamic dispatch and dependency injection, where imports under-approximate runtime coupling. (iv) Fixed-size commit windows put some snapshots years in the past (Requests, 2013). (v) We evaluated heuristic descriptions only, not LLM-written ones. (vi) The security rules are syntactic pattern matches, useful for tagging and routing but not a vulnerability scanner, and they report findings in tests.

Limitations of the proof of concept. It supports Python only, re-parses all files on every scan, resolves imports but not calls, and uses import-level rather than symbol-level edges. Requirement linking is lexical. We have not yet measured end-to-end effects on agent task success.

8 Future Work

- **Languages and edges.** A tree-sitter front end, call-level and symbol-level concepts, and typed edges for inheritance, data access and configuration.
- **Parse caching.** Cache per-file analysis by content hash, so that re-scan cost scales with the change rather than the repository.
- **Agent-in-the-loop evaluation.** Measure task success, tokens and reviewer effort for agents given category-aware context packs, versus top- k chunks and versus direct file exploration, in the spirit of [8].
- **Learned fusion.** Learn per-category weights for lexical, embedding, graph-distance and community signals from co-change history.
- **Global questions.** LLM-written subsystem summaries, and a GraphRAG-style map-reduce over them for architecture-level questions [4].
- **Serving.** An MCP server that exposes the bundle's navigation and context packs as tools, while keeping the bundle as the source of truth.
- **Profile governance.** Stabilise okf-code with more producers and consumers, including cross-repository links for multi-service systems.

9 Conclusion

Coding agents need context that is structural, current, and trustworthy. Rhizome publishes that context as an Open Knowledge Format bundle that lives with the code. Structure comes from a parser. Subsystems come from the Leiden algorithm, with its connectivity guarantee. An OKF code profile preserves dependency semantics in plain markdown. A co-evolution protocol makes knowledge updates part of ordinary code review. On six real repositories, scans are fast, deterministic and incremental, Leiden avoided the disconnected subsystems that Louvain occasionally produced,

and the import graph proved a much better relatedness signal than folder layout and complementary to lexical similarity. The resulting knowledge is readable by people, navigable by agents, and portable across tools, and it stays correct because every change that alters the code also updates it.

References

1. V. A. Traag, L. Waltman, N. J. van Eck. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* 9, 5233 (2019). arXiv:1810.08473.
2. V. D. Blondel, J.-L. Guillaume, R. Lambiotte, E. Lefebvre. Fast unfolding of communities in large networks. *J. Stat. Mech.* P10008 (2008).
3. M. E. J. Newman, M. Girvan. Finding and evaluating community structure in networks. *Phys. Rev. E* 69, 026113 (2004).
4. D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, J. Larson. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130 (2024).
5. P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS* (2020).
6. S. McVeety, A. Hormati. Introducing the Open Knowledge Format. Google Cloud Blog, June 12, 2026. Specification: github.com/GoogleCloudPlatform/knowledge-catalog/tree/main/okf.
7. A. Karpathy. LLM Wiki. GitHub gist, gist.github.com/karpathy/442a6bf555914893e9891c11519de94f (n.d.).
8. M. Vogel, F. Meyer-Eschenbach, S. Kohler, E. Grünwald, F. Balzer. Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP. arXiv:2603.27277 (2026).
9. Repowise. Leiden communities vs PageRank for code hotspots. repowise.dev/blog/concepts/leiden-communities-vs-pagerank-for-code-hotspots (accessed September 2026).
10. G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *SIGIR* (2009).
11. T. Zimmermann, P. Weißgerber, S. Diehl, A. Zeller. Mining version histories to guide software changes. *ICSE* (2004).
12. Microsoft. GraphRAG documentation. microsoft.github.io/graphrag (accessed September 2026).
13. S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3(4) (2009).
14. L. Page, S. Brin, R. Motwani, T. Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Stanford InfoLab Technical Report (1999).
15. S. Mancoridis, B. S. Mitchell, Y. Chen, E. R. Gansner. Bunch: A clustering tool for the recovery and maintenance of software system structures. *ICSM* (1999).
16. V. Tzerpos, R. C. Holt. ACDC: An algorithm for comprehension-driven clustering. *WCRE* (2000).

© 2026 Venkat Malviya. All rights reserved. This paper, the OKF code profile and the Rhizome software are the author's original work. Reproduction or use of the design or implementation, in whole or in part, requires the author's prior written permission. Evaluation artefacts: `eval/run_eval.py`, `eval/results.json`, `eval/results_scale.json`.